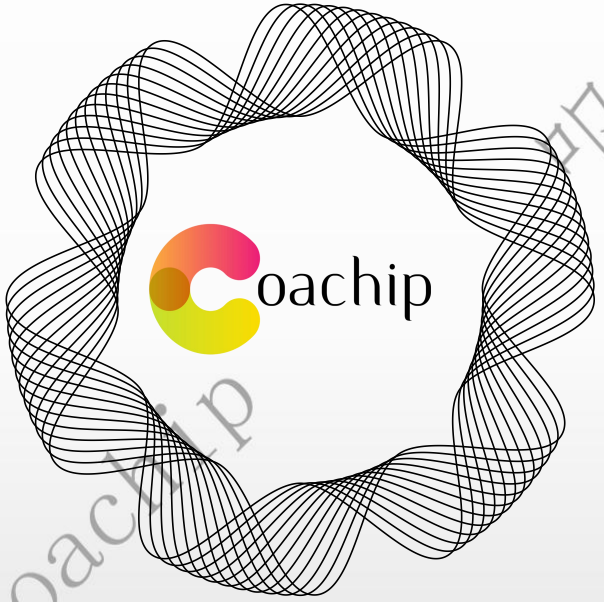


SV覆盖策略及覆盖组

叩特电子Coachip

课程内容及目标

课程内容：

- 覆盖策略
- 验证的完备性
- 定义和使用covergroup
- 在类里定义covergroup
- covergroup的采样触发
- 使用事件触发

叩特电子Coachip

收集信息而非数据

典型的例子就是FIFO

如何确定一个1K深度的FIFO是否被全面测试?

如果要测量读写index中的数据，组合可能有几百万种;

如果能使FIFO从空变满，再由满变空，那么就已经覆盖了几乎所有情形。

所以，我们收集的是信息，而不是具体数据。

叩特电子Coachip

只测量需要的内容

- 验证工程师需要懂得，在使能覆盖率收集时，这一特性会降低很大的仿真性能。
- 由于收集功能覆盖率数据的**开销**很大，所以应该只测量你会用来分析并且改进测试的那部分数据。
- 同时也需要设定合理的覆盖率采样的**事件**，一方面提升采样效率，一方面也可以降低收集覆盖率的开销。

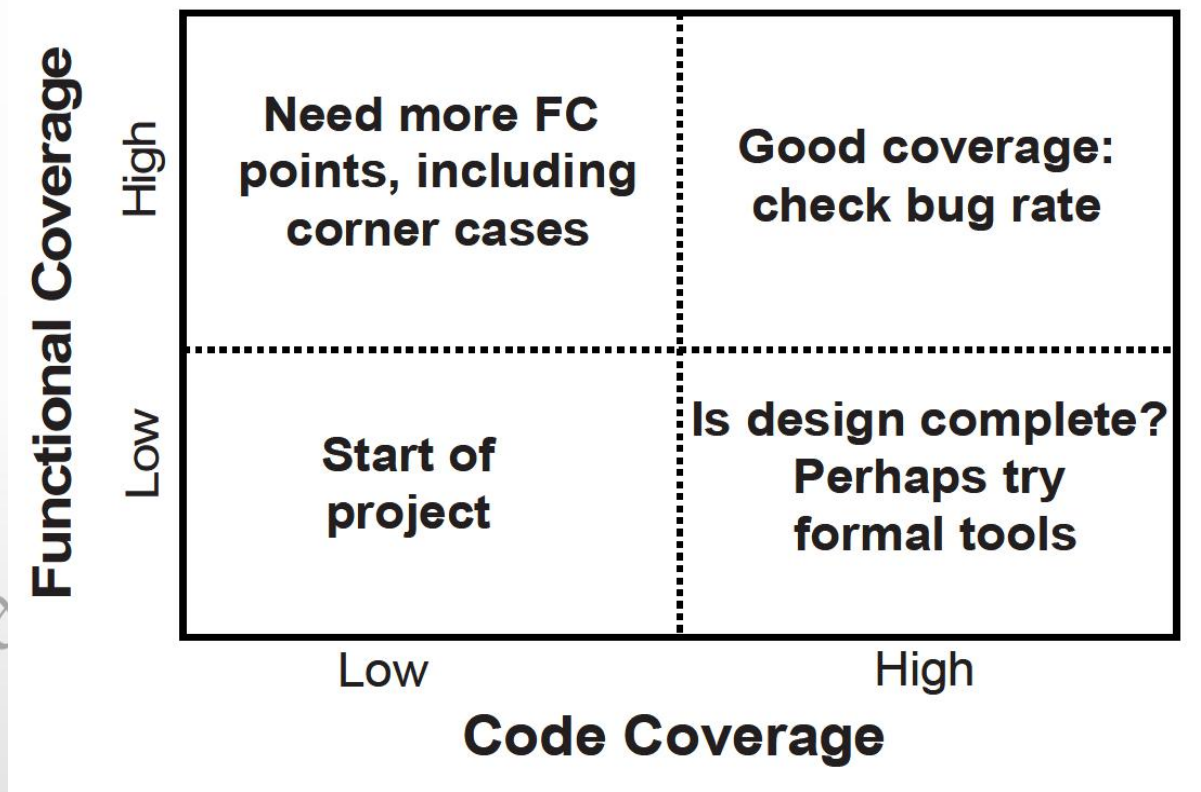
叩特电子Coachip

验证的完备性

- 完备的覆盖率测量结果和漏洞增长曲线，可以帮助确认设计是否被完整地验证过。
- 如果功能覆盖率高但代码覆盖率低，这说明验证计划不完整，测试没有执行设计的所有代码。
- 如果代码覆盖率高但功能覆盖率低，这说明即使测试平台很好地执行了设计的所有代码，但是测试还是没有把设计定位到所有感兴趣的状态上。
- 你的目标是同时**驱动**高的代码覆盖率和功能覆盖率。

叩特电子Coachip

验证的完备性



叩特电子Coachip

SV的覆盖组 covergroup—概述

- 覆盖组（covergroup）与类相似，一次定义后可多次实例化。
- covergroup可以包含一个或者多个coverpoint，且全都在同一时间采集。
- covergroup可以定义在类中，也可以定义在interface或者module中。
- covergroup可以采样任何可见的变量，例如程序变量、接口信号或者设计端口。

叩特电子Coachip

SV的覆盖组 covergroup—概述

- 一个类里可以包含多个covergroup。
- 当你拥有多个独立的covergroup时，每个covergroup可以根据需要自行使能或者禁止。
- 每个covergroup可以定义单独的触发采样事件，允许从多个源头收集数据。
- covergroup必须被例化才可以用来收集数据。

叩特电子Coachip

定义和使用covergroup

```
class Transaction;
  rand bit [31:0] data;
  rand bit [ 2:0] port; // Eight port numbers
endclass

program automatic test(busifc.TB ifc);
  covergroup CovPort;
    coverpoint ifc.port;   // Measure coverage
  endgroup
  initial begin
    Transaction tr;
    CovPort ck;
    ck = new(); // Instantiate group
    tr = new();
    repeat (32) begin         // Run a few cycles
      assert(tr.randomize);   // Create a transaction
      ifc.port <= tr.port;    // and transmit
      ifc.data <= tr.data;    // onto interface
      ck.sample(); // Gather coverage
      @ifc.clk;    // Wait a cycle
    end
  end
endprogram
```

叩特电子Coachip

在类里定义covergroup

```
class Transaction;
  logic [31:0] data;
  logic [ 2:0] port;
endclass

class Transactor;
  Transaction tr;
  mailbox mbx_in;
  virtual busifc ifc;

  covergroup CovPort;
    coverpoint tr.port;
  endgroup

  function new(mailbox mbx_in, virtual busifc ifc);
    CovPort = new();
    this.mbx_in = mbx_in;
    this.ifc = ifc;
  endfunction

 task main;
    begin
      mbx_in.get(tr);
      ifc.port <= tr.port;
      ifc.data <= tr.data;
      CovPort.sample();
    end
  endtask
endclass
```

叩特电子Coachip

使用类定义的covergroup

```
program automatic test(busifc ifc);

  initial begin
    Transaction tr;
    mailbox mbx_in;
    Transactor tr_c;

    tr = new();
    mbx_in = new();
    tr_c = new(mbx_in,ifc);

    repeat (32) begin         // Run a few cycles
      assert(tr.randomize);   // Create a transaction
      mbx_in.put(tr);
      tr_c.main(); // send driver and gather coverage
      @ifc.clk;    // Wait a cycle
    end
  end
endprogram
```

叩特电子Coachip

covergroup的采样触发

- covergroup由采样的数据和数据被采样的事件构成。当这两个条件都准备好以后，测试平台便会触发covergroup。
- 这个过程可以通过直接使用sample()函数完成，也可以在covergroup中采样阻塞表达式或者使用wait或@实现在信号或事件上的阻塞。
- 如果想在代码中显式地触发covergroup采样，或者不存在采样时刻的信号或事件，又或者一个covergroup被例化为多个实例需要单独触发，那么可以使用sample()方法。
- 如果你想借助已有的事件或者信号触发covergroup，可以在covergroup声明中使用阻塞语句。

即特电子Coachip

使用事件触发

```
event trans_ready;
covergroup CovPort @(trans_ready);
    coverpoint ifc.cb.port;
endgroup
```

与直接调用sample()相比，使用事件触发的好处在于你能够借助已有的事件，来决定何时开始收集数据。

叩特电子Coachip

练习一：

```
class Transaction;
  rand bit [31:0] data;
  rand bit [ 2:0] port; // Eight port numbers
endclass

program automatic test(busifc.TB ifc);
  covergroup CovPort;
    coverpoint ifc.port;   // Measure coverage
  endgroup
  initial begin
    Transaction tr;
    CovPort ck;
    ck = new(); // Instantiate group
    tr = new();
    repeat (32) begin         // Run a few cycles
      assert(tr.randomize);   // Create a transaction
      ifc.port <= tr.port;    // and transmit
      ifc.data <= tr.data;    // onto interface
      ck.sample(); // Gather coverage
      @ifc.clk;    // Wait a cycle
    end
  end
endprogram
```

叩特电子Coachip

练习二

```
class Transaction;
   rand logic [31:0] data;
   rand logic [ 2:0] port;
endclass

class Transactor;
    Transaction tr;
    mailbox mbx_in;
    virtual busifc ifc;

    covergroup CovPort;
      coverpoint tr.port;
    endgroup

    function new(mailbox mbx_in, virtual busifc ifc);
      CovPort = new();
      this.mbx_in = mbx_in;
      this.ifc = ifc;
    endfunction

   task main;
     begin
       mbx_in.get(tr);
       ifc.port <= tr.port;
       ifc.data <= tr.data;
       CovPort.sample();
     end
    endtask
endclass
```

叩特电子Coachip

练习二

```
program automatic test(busifc ifc);

  initial begin
    Transaction tr;
    mailbox mbx_in;
    Transactor tr_c;

    tr = new();
    mbx_in = new();
    tr_c = new(mbx_in,ifc);

    repeat (32) begin          // Run a few cycles
      assert(tr.randomize);    // Create a transaction
      mbx_in.put(tr);
      tr_c.main(); // send driver and gather coverage
      @ifc.clk;    // Wait a cycle
    end
  end
endprogram
```

叩特电子Coachip

Q&A

昂特电子Coachip